

Dataflow Optimization within High-Level Synthesis

Milestone 1: Technical Understanding and Research Framing

Electronics Engineering
Hochschule Hamm-Lippstadt

Problem Statement and Motivation

- Designing large ASICs manually at RTL level is no longer practical due to growing design complexity.
- High-Level Synthesis (HLS) automates the conversion: *behavioral description* $\rightarrow$ *RTL circuit*. The quality depends directly on how well the underlying Data-Flow Graph (DFG) is optimized.
- Two existing approaches, each with a fundamental weakness:
 - *Flat synthesis* – good quality but slow; does not scale to large designs.
 - *Hierarchical synthesis* – fast but poor quality; misses fine-grained optimizations across boundaries.
- **Core problem:** no existing method simultaneously achieves good solution quality *and* short synthesis time for large hierarchically structured designs.
- **Relevance for embedded systems:**
 - DSP and image-processing chips are central to embedded devices.
 - Power reduction is critical for battery-powered systems.
 - Short design cycles (weeks) demand fast synthesis tools.
 - The paper reports up to $6.7\times$ power reduction over an area-optimized baseline
 - very significant for resource-constrained hardware.

Core Technical Contribution

- **Hierarchical DFG as input:** the design is represented as an arbitrarily deep hierarchy of DFGs. Each hierarchical node is realized by an RTL module (simple or complex, e.g., an FFT or filter).
- **Iterative improvement:** start from an initial solution and repeatedly apply one of four *moves* until no further improvement is possible:
 - **Replace** – substitute an RTL module with a functionally equivalent but better-suited library module.
 - **Resynthesis** – rebuild a module's internals under relaxed timing constraints derived from available slack.

- **Merge** – combine two separate RTL modules into one shared module to reduce area.
- **Split** – decompose a shared module back into two to open new optimization opportunities (escaping local minima).
- **Key insight:** use the global view of the hierarchy to find slack that is invisible when optimizing one block in isolation; propagate that slack downward to drive fine-grain resynthesis at lower levels.
- **Simultaneous objectives:** the framework optimizes for area *or* power under a throughput constraint, and also selects supply voltage V_{dd} and clock period.

Central Algorithm and Formalism

Data-Flow Graph

- $G = (V, E)$: nodes V are arithmetic operations; directed edges E encode data dependencies.
- Edge $(u, v) \in E$: operation v cannot start until u has completed.
- *Hierarchical node*: a node whose behavior is itself described by a sub-DFG and realized by an RTL module.

Scheduling

Start time of operation v under a given resource binding:

$$t(v) = \max_{(u,v) \in E} [t(u) + d(u)] \quad (1)$$

where $d(u)$ is the delay in clock cycles of operation u . Ordering constraints on shared units are modeled as extra edges, reducing scheduling to a longest-path computation.

Slack

Spare cycles available to operation n before a deadline is violated:

- slack = 0: must use a fast (high-power) unit.
- slack > 0: can replace with a slower, lower-power unit without violating the deadline.

Top-Level Iterative Improvement Algorithm

Input: hierarchical DFG G , module library L ,
 objective (area or power), constraint C
 Output: optimized RTL circuit

```
foreach Vdd in candidate voltages:
  foreach clk in candidate clock periods:
    solution = Initial_solution(G, L)
    while improvement possible:
```

```

    for i = 0 to MAX_MOVES:
        try Replace / Resynthesis move
        try Merge move
        if no gain: try Split move
        apply best move sequence
    track best solution seen
return best solution

```

Assumptions and Scope Limitations

- **Data-flow intensive only:** targets designs with arithmetic-dominated behavior (DSP, image processing). Control-flow intensive behaviors (`if/else`, variable-bound loops) are explicitly out of scope.
- **Fixed sampling period:** the throughput constraint T_s is given as input; the algorithm minimizes area or power *subject to* T_s , not the latency itself.
- **Pipelining is pre-synthesis:** the algorithm does not pipeline behavioral descriptions itself; already-pipelined behaviors may be provided as input.
- **Known module internals:** Resynthesis moves cannot be applied to modules whose internal description is unavailable (e.g., hard IP cores).
- **Power model accuracy:** the switched-capacitance matrix (SCM) model approximates power at the functional-unit level; detailed interconnect and layout effects are not captured until post-synthesis.

Runtime, Scalability, and Resource Aspects

- **Scheduling** (longest-path): $O(|V| + |E|)$ – negligible cost.
- **RTL module merging:** at least as hard as graph isomorphism (NP-hard); a heuristic based on iterative node association is used instead of an exact algorithm.
- **Functional isomorphism checking:** exact solution feasible only for DFGs with $\lesssim 100$ nodes; heuristics used for larger sub-graphs.
- **Synthesis time:** hierarchical synthesis is $\approx 4\times$ faster than flat synthesis on the tested benchmarks (12–217 nodes when flattened).

References

- [1] G. Lakshminarayana and N.K. Jha, “High-Level Synthesis of Power-Optimized and Area-Optimized Circuits from Hierarchical Data-Flow Intensive Behaviors,” *IEEE Trans. Comput.-Aided Des. Integr. Circuits Syst.*, vol. 18, no. 3, pp. 265–281, Mar. 1999.